

# Visit Ethiopia Agentic RAG Chatbot

*Complete Beginner-to-Pro Manual: Setup, Dependencies, Ingestion, Re-indexing & Serving*

---

## 1. Project Overview & Architecture

The **Visit Ethiopia RAG Chatbot** is an autonomous artificial intelligence assistant designed to answer travel queries using data crawled directly from official Ethiopian national and regional tourism portals. It uses a Retrieval-Augmented Generation (RAG) pipeline combining web crawlers, HuggingFace sentence embeddings, a Chroma vector database, hybrid BM25 search, Google Gemini LLMs with automatic key rotation, and local Ollama fallback models.

## 2. System Prerequisites

- **Python 3.10+** (Recommended: Python 3.10.11)
- **Google Chrome Browser** (Required for Selenium dynamic web crawler)
- **Git** (For project version control)
- **Google Gemini API Key(s)** (Free from Google AI Studio)
- **(Optional) Ollama** (If using local offline LLM like *mistral:7b*)

## 3. Step-by-Step Environment Setup

Open terminal and navigate to the project directory:

```
cd "c:\Users\liche\Desktop\RAG-Chatbot\RAG-Chatbot-from-web-data"
```

Create Virtual Environment:

```
python -m venv venv
```

Activate Virtual Environment:

```
# Windows (PowerShell)
.\venv\Scripts\Activate.ps1

# Windows (Command Prompt)
venv\Scripts\activate.bat

# Linux / macOS
source venv/bin/activate
```

## 4. Installing Required Libraries (Direct Commands)

Instead of using requirements.txt, install all necessary libraries directly using the two standard pip install commands below:

### Command 1: Core Data Science & Utility Packages

```
pip install numpy pandas scikit-learn scikit-image matplotlib seaborn tqdm
```

### Command 2: Core LangChain, Gemini, RAG, Web & Serving Stack

```
pip install -U langchain langchain-core langchain-community langchain-google-genai langchain-chroma google-generativeai rank-bm25 flask flask-cors gunicorn chromadb sentence-transformers beautifulsoup4 bs4 html2text requests selenium webdriver-manager python-dotenv tqdm markdown streamlit pypdf typing-extensions pydantic protobuf
```

## 5. Environment Configuration (.env Setup)

Create a file named .env inside the RAG-Chatbot-from-web-data/chatbot directory with the following contents:

```
# Google Gemini API Keys (Supports multi-key round-robin rotation)
GOOGLE_API_KEY_1=AIzaSyACRPPvdOptcazzjUmVCJV2Sd2x88mLBWs
GOOGLE_API_KEY_2=AIzaSyChm5oonrV6RU3_Zqo0tL4JpKteLeN3w_Q

# Local Ollama LLM Fallback
OLLAMA_MODEL=mistral:7b
OLLAMA_BASE_URL=http://localhost:11434

# API Security & Limits
CHATBOT_API_KEY=LY0qxG-9KM8CcF0tZF9iWr2h0l0vui00YPimW_0g90UfHvdQMsh8dbRqbbmzwQCe
REQUIRE_API_KEY=false
ALLOWED_ORIGINS=https://visitethiopia.et,http://localhost:8501
REQUEST_TIMEOUT_SECONDS=60
ANONYMIZED_TELEMETRY=False
CRAWL_INTERVAL_HOURS=24
```

## 6. Data Ingestion & Vector Indexing (3 Methods)

First navigate into the chatbot module directory: `cd chatbot`

### Method A: One-Off Manual CLI Ingestion

Crawls configured official websites and builds the ChromaDB index in chatbot/data/chroma:

```
python ingest.py
```

To target specific websites:

```
python ingest.py --url https://visitethiopia.et/ --url https://visitoromia.org/
```

## Method B: 24-Hour Background Automated Scheduler

Runs a background daemon that updates vector embeddings automatically every 24 hours:

```
python scheduler.py
```

## Method C: On-Demand API Re-indexing Endpoint (/admin/reindex)

When the Flask API server is running, administrators can trigger website crawling and vector re-indexing on demand via HTTP POST request to the /admin/reindex endpoint:

```
# Trigger Re-indexing via cURL command:
curl -X POST http://localhost:5000/admin/reindex \
  -H "Content-Type: application/json" \
  -H "X-API-Key: LY0qxG-9KM8Ccf0tZF9iWr2h0l0vui00YPimW_0g90UfHvdQMsh8dbRqbbmzWQCe" \
  -d '{"urls": ["https://visitethiopia.et/"]}'

# Expected HTTP 200 JSON Response:
{
  "status": "success",
  "message": "Reindexing finished",
  "success_sources": 1,
  "failed_sources": 0
}
```

## 7. Starting and Running the Services

Make sure you are inside chatbot/ directory with virtual environment active.

### Start Service 1: Flask REST API Backend Server

```
python api_server.py
```

- Server URL: <http://localhost:5000>
- Health Check: <http://localhost:5000/health>
- Question Answering Endpoint: POST <http://localhost:5000/ask>

### Start Service 2: Streamlit Interactive Web Application

```
streamlit run app.py
```

- Browser Access: Opens automatically at <http://localhost:8501>

## 8. Testing & Verification Summary

| Step | Action / Component   | Execution Command           | Success Output             |
|------|----------------------|-----------------------------|----------------------------|
| 1    | Activate Virtual Env | .\venv\Scripts\Activate.ps1 | (venv) prompt visible      |
| 2    | Install Libraries    | pip install -U ...          | Packages installed cleanly |
| 3    | Data Ingestion       | python ingest.py            | Chroma database created    |
| 4    | API Re-indexing      | POST /admin/reindex         | Status success JSON        |
| 5    | Flask API Backend    | python api_server.py        | HTTP 200 on /health        |
| 6    | Streamlit Web UI     | streamlit run app.py        | Opens at localhost:8501    |